

Java Collections framework

Collections

It is basically a group of objects while the collection framework is a toolbox for managing a group of objects

The collection contains:

- List
- Map
- Queue
- Set

While Array can store data of same data type , has a fixed size and does not contain ready made methods while collections can store dynamic dataset and it does not have a fixed memory size and have various ready made methods like searching, sorting etc. to perform various operations

Core Interfaces in the Collections Framework

Here are some common interfaces, along with their classes:

Interface	Common Classes	Description
List	ArrayList , LinkedList	Ordered collection that allows duplicates
Set	HashSet , TreeSet , LinkedHashSet	Collection of unique elements
Map	HashMap , TreeMap , LinkedHashMap	Stores key-value pairs with unique keys

Note:

Use List classes when you care about order, you may have duplicates, and want to access elements by index.

Use Set classes when you need to store unique values only.

Use Map classes when you need to store pairs of keys and values, like a name and its phone number.

List

- It is an interface
- It is ordered dataset and it allows duplicates in it.

- We can access the list by its indexes
- Since a list is an interface so We cannot access it directly ,we use a class to create an object of the list and then access/use it
Instead, you use a class that implements the `List` interface, such as:
 - [ArrayList](#) - like a resizable array with fast random access
 - [LinkedList](#) - like a train of cars you can easily attach or remove

Tip: Use `List` when you care about order, you may have duplicates, and want to access elements by index.

Generics is used for type safety

We can also create a class and through the list can also use the list of objects to perform various operations.

Common List Methods

Method	Description
<code>add()</code>	Adds an element to the end of the list
<code>get()</code>	Returns the element at the specified position
<code>set()</code>	Replaces the element at the specified position
<code>remove()</code>	Removes the element at the specified position
<code>size()</code>	Returns the number of elements in the list

List vs. Array

Array	List
Fixed size	Dynamic size
Faster performance for raw data	More flexible and feature-rich
Not part of Collections Framework	Part of the Collections Framework

Java ArrayList

The `ArrayList` in Java is a resizable array which may have duplicates at the same time the elements can be added or removed from anywhere, and it is present in the `java.util` package. Elements in the list are Objects since `String` is itself a class its objects are created ,while if we have to use other primitive types then we use the wrapper class

After Java10 , the List can also be declared as var but for the clarity the developers prefer to refer it as List only

Java Linked List

In the linked List the ,elements are added to the containers and the containers are linked to the other container while in the array ,if it is not big enough new one is created and replaces the older one ,While the ArrayList is used to store the data ,the linked list is used to manipulate the data

Method	Description
<code>addFirst()</code>	Adds an element to the beginning of the list
<code>addLast()</code>	Add an element to the end of the list
<code>removeFirst()</code>	Remove an element from the beginning of the list
<code>removeLast()</code>	Remove an element from the end of the list
<code>getFirst()</code>	Get the element at the beginning of the list
<code>getLast()</code>	Get the element at the end of the list

Sorting an an ArrayList or LinkedList we can use sort method to do it in ascending order while we use reverseOrder() to do it in descending order.

Set

It is an interface

It is unordered (which means you will not have indexes like in list) and does not allow duplicates Set is mostly used in Spring Security for managing role and things like that

Common classes that implement Set :

- [HashSet](#) - fast and unordered
- [TreeSet](#) - sorted set
- [LinkedHashSet](#) - ordered by insertion

Tip: Use a Set when you need to store unique values only.

Common Set Methods

Method	Description
<code>add()</code>	Adds an element if it's not already in the set
<code>remove()</code>	Removes the element from the set

Method	Description
<code>contains()</code>	Checks if the set contains the element
<code>size()</code>	Returns the number of elements
<code>clear()</code>	Removes all elements

Set vs. List

List	Set
Allows duplicates	Does not allow duplicates
Maintains order	Does not guarantee order
Access by index	No index-based access

Java HashSet

It is a collection of elements where the elements are unique and it is also the part of `Java.util` package.

Note: Duplicate values are not counted - only unique elements are included in the size.

Java TreeSet

The `TreeSet` is also a collection where the elements are stored in an ordered manner uniquely and it implements the `Set` interface.

Unlike `HashSet`, which has no order, `TreeSet` keeps its elements sorted automatically.

HashSet vs TreeSet

Feature	<code>HashSet</code>	<code>TreeSet</code>
Order	No guaranteed order	Sorted (natural order)
Duplicates	Not allowed	Not allowed
Performance	Faster (no sorting)	Slower (due to sorting)

Java LinkedHashSet

The `linkedHashSet` is also a collection that implements the `Set` interface and it stores the unique elements in the ordered they were inserted

Tip: Use `HashSet` when you only care about uniqueness and speed.

Use `LinkedHashSet` when order matters.

Tip: Use `LinkedHashSet` when you want a set that does not allow duplicates *and* keeps the original insertion order.

HashSet vs LinkedHashSet

Feature	HashSet	LinkedHashSet
Order	No guaranteed order	Insertion order preserved
Duplicates	Not allowed	Not allowed
Performance	Faster	Slightly slower (due to order tracking)

Map

It is an interface which stores data in the form of key value pair and then it does not have duplicate key, while the value can be duplicate. One key will map at most one value.

A `Map` is useful when you want to associate a key (like a name or ID) with a value (like an age or description).

When we have to return data in the form of json then mostly we use map.

Note: If the datatype that has to be used cannot be used as a primitive data type then we use wrapper class for that.

Tip: Use a `Map` when you want to associate values with unique keys, like storing user IDs with names.

Common Map Methods

Method	Description
<code>put()</code>	Adds or updates a key-value pair
<code>get()</code>	Returns the value for a given key
<code>remove()</code>	Removes the key and its value
<code>containsKey()</code>	Checks if the map contains the key
<code>keySet()</code>	Returns a set of all keys

Map vs. Set vs. List

Feature	List	Set	Map
Duplicates allowed?	Yes	No	Keys: No Values: Yes
Stores key-value pairs?	No	No	Yes
Maintains order?	Yes	No (unless using TreeSet or LinkedHashSet)	No (unless using TreeMap or LinkedHashMap)

Common classes that implement Map :

- `HashMap` - fast and unordered
- `TreeMap` - sorted by key
- `LinkedHashMap` - ordered by insertion

To traverse through the collection we can use advanced for loop, for loop and Iterators.

Java HashMap

A Hashmap is basically a collection that implements the Map Interface, it uses key to retrieve the value and works in the format of key,value pair

Where it is also the part of java.util package.

A `HashMap` can store many different combinations, such as:

- `String` keys and `Integer` values
- `String` keys and `String` values

Loop Through a HashMap

Loop through the items of a `HashMap` with a **for-each** loop.

Note: Use the `keySet()` method if you only want the keys, and use the `values()` method if you only want the values:

Example

```
// Print keys
for (String i : capitalCities.keySet()) {
    System.out.println(i);
}
```

Java Tree Map

The `TreeMap` is the collection that stores the values in key,value pair in the sorted order by the key ,

It also uses `java.util` package and implements `Map` interface

Tip: Unlike `HashMap` , which does not maintain order, `TreeMap` keeps its keys sorted.

Note: Use the `keySet()` method if you only want the keys, and use the `values()` method if you only want the values:

Example

```
// Print keys
for (String i : capitalCities.keySet()) {
    System.out.println(i);
}
```

```
// Print values
for (String i : capitalCities.values()) {
    System.out.println(i);
}
```

```
/ Print keys and values
for (String i : capitalCities.keySet()) {
    System.out.println("key: " + i + " value: " + capitalCities.get(i));
}
```

Tree Map vs HashMap

Feature	HashMap	TreeMap
Order	No guaranteed order	Sorted by keys
Null Keys	Allows one null key	Does not allow null keys
Performance	Faster (no sorting)	Slower (maintains sorted order)

Tip: Use `HashMap` for performance, and `TreeMap` when you need sorted keys.

Java Linked HashMap

The `LinkedHashMap` is used to store the keys and values and it keeps the values in the same order that they are inserted ,uses the `java.util` package and implements `Map` interface.

Tip: Use `LinkedHashMap` when you want predictable iteration order (insertion order).

Note: The `size()` only counts unique keys. If a key is added more than once, only the latest value is kept.

The duplicate keys are ignored if they contain different values and same key, it uses the latest inserted value and counts it as one key.

You can loop through a `LinkedHashMap` using a for-each loop. Use:

- `keySet()` to get all keys
- `values()` to get all values

HashMap vs LinkedHashMap

Feature	HashMap	LinkedHashMap
Order	No guaranteed order	Insertion order preserved
Performance	Faster for random access	Slightly slower due to ordering
Duplicates	Keys must be unique	Keys must be unique

Tip: Use `LinkedHashMap` when you want the map to remember the order in which entries were added.